Blobwar – Stratégies de jeu

MAOUDE Samir MONTAURIOL Nicolas

May 2, 2026

Abstract

Ce rapport présente les différentes stratégies que nous avons programmées pour le jeu Blobwar: une stratégie gloutonne, minimax, alpha–beta, puis des versions parallèles sur CPU. Nous expliquons aussi les améliorations que nous avons ajoutées progressivement: bitboards, tables de transposition, hachage de Zobrist, recherche avec mémoire et anticipation du coup adverse.

Le tableau de bord interactif des expériences est disponible ici:

https://makh.fr/blobwar/

1 Règles utiles

Blobwar se joue sur un plateau 8×8 contenant éventuellement des trous. Un coup consiste soit à cloner un blob sur une case voisine, soit à effectuer un saut à distance deux. Après le déplacement, les blobs adverses adjacents à la case d'arrivée sont convertis.

Les points auxquels il faut faire attention sont les suivants:

- Si aucun coup légal n'est possible, le joueur passe son tour. Le moteur ne doit donc pas supposer qu'un nœud sans coup est nécessairement une défaite immédiate.
- Une partie peut devenir nulle si les joueurs répètent une situation ou si aucun progrès n'est réalisé. Les stratégies doivent donc éviter les cycles inutiles lorsque le gain est accessible.
- Le tournoi lance alternativement l'algorithme comme joueur 1 puis comme joueur 2. La fonction d'évaluation et la génération des coups sont donc écrites du point de vue du joueur courant, sans supposer que notre couleur est toujours la même.

2 Organisation du code

Le code des stratégies suit une interface commune: une stratégie reçoit l'état courant du plateau, énumère les coups légaux, évalue les états accessibles puis publie le meilleur coup trouvé. Les premières versions conservent la représentation historique du plateau, tandis que les versions optimisées passent à une représentation bitboard et à une recherche avec mémoire.

Avec les bitboards, le plateau est représenté par quelques masques 64 bits: un masque pour nos blobs, un masque pour les blobs adverses, et un masque pour les cases interdites. Les déplacements possibles sont stockés dans des tableaux fixes, donc pour une case donnée on récupère directement les destinations de clonage ou de saut, puis on filtre avec les masques rouge, bleu et trous. Comme le plateau a une taille fixe, cette représentation permet de calculer les cases accessibles par opérations bit à bit en $O(1)$.

Code	Principe	Rôle dans le projet
`Glouton`	Évaluation de chaque coup légal à profondeur 1	Version simple de départ
`Minimax`	Exploration exhaustive à profondeur fixée	Recherche simple à profondeur fixe
`Alpha-beta`	Coupures par bornes α et β	Recherche séquentielle efficace
`NMP + TT`	Négamax, bitboards, Zobrist, table de transposition	Évite de recalculer les mêmes positions
`Mémoire entre les tours`	Mémoire conservée entre les recherches	Réutilise les recherches précédentes
`Anticipation`	Anticipation du coup adverse pendant son temps de réflexion	Gain de temps entre deux tours
`Lazy SMP`	Plusieurs recherches CPU partageant la même TT	Recherche sur plusieurs cœurs

Table 1: Résumé des principales idées présentes dans le code.

3 Fonction d'évaluation

La fonction d'évaluation doit être rapide, car elle est appelée à toutes les feuilles de l'arbre de recherche. La première version utilise principalement la différence de matériel:

$$\boxed{\mathbf{eval}(\mathbf{s}, \mathbf{p}) = \#\mathbf{blobs_p}(\mathbf{s}) - \#\mathbf{blobs_{\bar{p}}}(\mathbf{s})}.$$

Avec la représentation bitboard, ce calcul est quasiment immédiat: il suffit de compter les bits à 1 dans le masque du joueur et dans le masque adverse. Cela rend l'heuristique très peu coûteuse, même lorsqu'elle est appelée sur des millions de feuilles.

Contrairement aux échecs, où la position des pièces peut complètement changer la valeur réelle du matériel, Blobwar a une mécanique plus directe: on clone, on saute, puis on convertit les cases voisines. Dans ce contexte, le nombre de blobs donne déjà une information très forte. Nos essais nous ont donc poussés à faire confiance à une heuristique simple, utilisée avec une recherche plus profonde, plutôt qu'à ajouter trop de règles écrites à la main.

Dans les versions suivantes, nous avons surtout gardé l'idée suivante: l'évaluation doit rester simple, car le gain principal vient du fait de chercher plus profond. Nous avons donc privilégié une fonction rapide, combinée avec un meilleur parcours de l'arbre: **alpha–beta**, **tri des coups**, **table de transposition** et **recherche parallèle**.

4 Algorithme glouton

La stratégie gloutonne énumère tous les coups légaux, applique chaque coup sur une copie du plateau, évalue l'état obtenu puis sélectionne le meilleur coup. Sa complexité est proportionnelle au nombre de coups légaux:

$$\boxed{\mathbf{O(n)}}$$

Où n **est le nombre moyen de coups possibles** dans une position.

Cette stratégie nous sert de point de départ. Elle est rapide mais myope: elle peut choisir une conversion immédiate qui donne ensuite un gros avantage à l'adversaire.

Listing 1: Sélection gloutonne dans `strategy_glouton.cc`

```
void Strategy::computeBestMove() {
    vector<movement> valid_moves;
    computeValidMoves(valid_moves);

    movement best_move(0, 0, 0, 0);
    Sint32 best_score = -10000;

    for (const auto& mv: valid_moves) {
        Strategy next(*this);
        next.applyMove(mv);

        Sint32 score = next.estimateCurrentScore();
        if (score > best_score) {
            best_score = score;
            best_move = mv;
        }
    }
    _saveBestMove(best_move);
}
```

5 Minimax

Minimax explore l'arbre de jeu jusqu'à une profondeur fixée. Le joueur courant choisit le coup qui donne le meilleur score final, tandis que l'adversaire choisit celui qui nous donne le pire score. Dans le code, l'implémentation est écrite sous forme négamax:

$$\boxed{\mathbf{score}(\mathbf{s}, \mathbf{d}) = \max_{\mathbf{m} \in \mathbf{coups}(\mathbf{s})} -\mathbf{score}(\mathbf{apply}(\mathbf{s}, \mathbf{m}), \mathbf{d} - \mathbf{1})}.$$

La version `strategy_minimax.cc` utilise une profondeur fixe. Cette approche est plus forte que le glouton mais devient rapidement coûteuse:

$$\boxed{\mathbf{O}(\mathbf{n}^{\mathbf{d}})}.$$

Elle est donc utile comme étape intermédiaire et comme comparaison avec les versions plus avancées.

Listing 2: Minimax écrit sous forme négamax dans `strategy_minimax.cc`

```
Sint32 Strategy::minimax(int depth) {
    if (depth == 0 || isEndGame()) {
        return estimateCurrentScore();
    }

    vector<movement> valid_moves;
    computeValidMoves(valid_moves);

    Sint32 best_score = -10000;
    for (const auto& mv: valid_moves) {
        Strategy next(*this);
        next.applyMove(mv);
        Sint32 score = -next.minimax(depth - 1);
        best_score = std::max(best_score, score);
    }
    return best_score;
}
```

6 Alpha–beta

Alpha–beta calcule le même résultat que minimax à profondeur égale, mais coupe les branches qui ne peuvent plus influencer la décision finale. La borne α représente le meilleur score déjà garanti au joueur maximisant, tandis que β représente la meilleure borne connue pour le joueur minimisant. Dès que $\alpha \geq \beta$, la branche courante est abandonnée.

À ce moment du projet, on savait déjà qu'on voulait aller vers une stratégie *stateful*, c'est-à-dire une stratégie qui garde des informations entre deux recherches: meilleur coup trouvé, table de transposition, ordre des coups, puis plus tard anticipation pendant le tour adverse. Pour éviter d'avoir deux codes séparés selon le joueur qui maximise ou minimise, nous avons donc réécrit la recherche en **négamax**. Le principe est que le score est toujours vu du point de vue du joueur courant; quand on descend d'un niveau dans l'arbre, on inverse simplement le signe du score. Cela simplifie le code et rend les ajouts suivants plus faciles à brancher.

L'intérêt pratique dépend fortement de l'ordre des coups. Les versions avancées trient donc les coups avec plusieurs sources d'information:

- coups de clonage avant certains sauts lorsque cela augmente la stabilité;
- meilleur coup issu de la **table de transposition**;
- score heuristique du coup à la racine;
- historique des coups qui ont provoqué des coupures.

L'iterative deepening complète alpha–beta: on cherche d'abord à profondeur 1, puis 2, etc. Cela donne rapidement un coup jouable et améliore l'ordre des coups pour les profondeurs suivantes.

Listing 3: Cœur simplifié de l'alpha–beta dans la recherche négamax

```
static Sint32 negamax(const Board &b, Uint16 player,
                      int depth, Sint32 alpha, Sint32 beta) {
    if (depth == 0) {
        return evaluate(b, player);
    }

    Sint32 best = -INF;

    for (const Move& mv : generateMoves(b, player)) {
        Board child = applyMove(b, mv);
        Sint32 score = -negamax(child, opponent(player),
                                depth - 1, -beta, -alpha);

        best = std::max(best, score);
        alpha = std::max(alpha, score);

        if (alpha >= beta) {
            break; // coupure alpha-beta
        }
    }
    return best;
}
```

7 Parallélisation CPU

7.1 Minimax parallèle

Pour paralléliser minimax, l'idée la plus simple est de répartir les coups possibles du premier niveau entre plusieurs threads CPU. Chaque thread calcule le score d'un sous-arbre, puis on garde le meilleur score à la fin. Cette répartition est naturelle, car les sous-arbres ne dépendent pas les uns des autres.

Le problème est que les threads partagent peu d'information. Ils explorent chacun leur partie de l'arbre, sans toujours profiter des coupures trouvées ailleurs. Cela reste intéressant pour mesurer le gain brut du parallélisme.

7.2 Alpha–beta parallèle

Alpha–beta est plus difficile à paralléliser. En séquentiel, l'algorithme devient efficace parce qu'un bon coup trouvé tôt permet de couper beaucoup de branches. Si on lance tous les coups en même temps, les threads commencent avec peu d'information et on peut perdre une partie de ces coupures.

Nous avons donc utilisé deux idées simples:

- chercher d'abord un premier coup pour obtenir une borne initiale;
- une table de transposition partagée entre threads, utilisée en lecture et écriture avec une politique de remplacement par profondeur.

La variante ***Lazy SMP*** lance plusieurs recherches alpha–beta qui partagent la même table de transposition. Si deux threads écrivent au même endroit, cela ne rend pas le coup invalide: on perd seulement une information qui aurait pu accélérer la recherche.

Nous avons volontairement gardé une synchronisation légère sur cette table. Dans ce contexte, la table de transposition sert surtout de boussole commune: elle aide les threads à retomber sur des coups déjà prometteurs et à éviter une partie du travail en double. On préfère donc accepter quelques écritures concurrentes ou quelques entrées moins fiables plutôt que de bloquer les threads avec des verrous trop coûteux. Le résultat final reste vérifié par la recherche alpha–beta; la table sert à orienter et accélérer la recherche, pas à décider seule du coup joué.

Listing 4: Parallélisation OpenMP

```
// Le premier coup tente de trouver une borne initiale.
scores[0] = -negamax(firstChild, opponent(me), depth - 1,
                  -INF, INF, 1, firstKey, rootTL, stop);

std::atomic<Sint32> sharedBound(scores[0]);

#pragma omp parallel
{
    ThreadLocal tl;
    initThreadLocal(tl);

#pragma omp for schedule(dynamic)
    for (int i = 1; i < moveCount; ++i) {
        Sint32 localBound = sharedBound.load(std::memory_order_relaxed);
        Board child = applyMove(b, rootMoves[i]);
        Sint32 score = -negamax(child, opponent(me), depth - 1,
                            -INF, -localBound, 1, childKey, tl, stop);

        Sint32 prev = sharedBound.load(std::memory_order_relaxed);
        while (score > prev) {
            if (sharedBound.compare_exchange_weak(prev, score)) {
                break;
            }
        }
    }
}
```

7.3 Version finale MM0

La version MM0 est la version que nous avons le plus travaillée. Elle regroupe les choix qui ont donné les meilleurs résultats pendant le projet. L'objectif était d'avoir une stratégie robuste sous limite de temps, tout en gardant suffisamment de logs pour comprendre ce qui se passe pendant les matchs.

Ses composants principaux sont:

- une représentation **bitboard** du plateau;
- une recherche **négamax** avec coupures **alpha–beta**;
- une **table de transposition Zobrist** remise dans un état contrôlé entre recherches;
- un **tri des coups** à la racine et dans les nœuds internes;
- une **parallélisation CPU** de la recherche racine;
- une **mémoire** qui garde de l'information utile entre les tours;
- une **anticipation du coup adverse** pendant son temps de réflexion;
- des **statistiques détaillées**: profondeur, NPS (nœuds par seconde), taux de hit TT, taux de cutoff, tri des coups et réussite de l'anticipation.

Cette version reprend les algorithmes demandés et ajoute les améliorations que nous avons testées ensuite. Sur le site, on compare plusieurs générations de cette famille (`NMP_TT_v1` et `v2`, `NMP_TT_SF_v1`, `v2`, `v3`, `v5`, `v7`). Cela permet de voir ce que chaque ajout apporte, au lieu de regarder seulement la dernière version.

7.4 Notes sur quelques variantes

La version `v3` est celle que nous avons gardée comme version de puissance brute: elle cherche vite et profond, avec peu de logique supplémentaire autour de l'ordre des coups. La version `v5` reprend cette base mais ajoute un tri des coups plus travaillé. En pratique, même pendant les tournois, cela nous a conduits à choisir l'algorithme selon la carte: `v3` était plus intéressante sur `standard`, alors que `v5` donnait de meilleurs résultats sur `x`.

La version `v3_15` vient d'une contrainte matérielle observée sur les machines de test. Les Intel Core i7-12700 utilisés à l'ENSIMAG exposent 20 threads logiques, mais tous ne sont pas équivalents. Un P-core est un cœur performant: sur cette machine, chaque P-core expose deux threads logiques grâce à l'Hyper-Threading. Ces deux threads sont des *siblings*: ils partagent le même cœur physique, donc ils ne valent pas deux vrais cœurs indépendants, mais ils restent intéressants pour occuper les unités de calcul quand un thread attend la mémoire. À côté, les E-cores sont des cœurs plus petits, faits pour consommer moins, mais moins adaptés à notre recherche qui lance beaucoup de calculs courts et intenses.

Concrètement, les 16 premiers threads logiques correspondent aux 8 P-cores avec leurs siblings, alors que les 4 derniers correspondent aux E-cores. Comme ces derniers donnaient moins de puissance utile et pouvaient perturber l'équilibrage du travail, nous avons préféré limiter la version au seuls P-cores. Parmi les 16 threads restants nous avons aussi exclu le *sibling* du thread principal (celui qui recherche le coup principal), afin de lui laisser un accès complet au coeur physique. La configuration utilise donc 15 threads de calcul.

La version `v7` teste une idée différente: charger une énorme base de coups pour orienter l'ordre de recherche. Cette base est construite à partir des coups observés dans des parties gagnées, puis stockée dans des fichiers JSON séparés par carte et par couleur. Dans le code, une position est repérée par ses bitboards rouge et bleu, puis la table renvoie jusqu'à trois coups candidats. Ces coups ne remplacent pas alpha–beta: ils sont simplement essayés tôt dans l'ordering, après le coup de la table de transposition et avant les heuristiques plus générales. Ce n'est donc pas une table de fin de partie exhaustive, mais une mémoire empirique de coups qui ont déjà bien fonctionné. Nous n'avons pas eu le temps de simuler assez de parties pour produire des tablse suffisament conséquentes mais la version était très prometteuse.

8 Optimisations

8.1 Bitboards

Les versions optimisées remplacent les tableaux par des bitboards 64 bits. Une case du plateau correspond à un bit. Les opérations de génération de coups, de conversion et de comptage utilisent alors des opérations bit à bit et des intrinsics comme `__builtin_popcountll`. Cela réduit fortement le coût des nœuds internes.

Listing 5: Représentation bitboard et fonctions de base

```
typedef Uint64 Bitboard;

static inline int index(int x, int y) { return y * 8 + x; }
static inline Bitboard mask(int s) { return ((Uint64)1 << s); }
static inline int next(Bitboard bb) {
    return __builtin_ctzll((unsigned long long)bb);
}

static inline Bitboard denied(const Board &b) {
    return b.red | b.blue | b.holes;
}

static inline Bitboard allowed(const Board &b) {
    return ~denied(b);
}
```

8.2 Tables de transposition

Une table de transposition mémorise les positions déjà rencontrées. Chaque entrée contient une clé, une profondeur, un score, un type de borne (`EXACT`, `LOWER`, `UPPER`) et parfois le meilleur coup trouvé. Lorsqu'une position est revue à profondeur suffisante, la table permet soit de retourner directement un score, soit de resserrer la fenêtre alpha–beta.

Listing 6: Insertion dans la table de transposition

```
static inline void storeTT(Uint64 key, int depth,
                           Sint32 value, int flag, const Move &best) {
    TTEntry &entry = transTable[key & (TT_SIZE - 1)];

    if (entry.flag != TT_EMPTY) {
        if (entry.key == key && entry.depth > depth) return;
        if (entry.key != key && entry.depth > depth) {
            TT_REJECTED_COLLISIONS++;
            return;
        }
    }

    entry.key = key;
    entry.depth = depth;
    entry.value = value;
    entry.flag = flag;
    entry.best = best;
}
```

8.3 Hachage de Zobrist

Le hachage de Zobrist associe une valeur aléatoire 64 bits à chaque couple (joueur, case), puis combine ces valeurs par XOR. L'intérêt est que la clé peut être mise à jour incrémentalement après un coup, au lieu de recalculer tout le plateau.

Dans notre code, cette clé sert directement à retrouver rapidement une position dans la table de transposition.

Une optimisation possible, que nous n'avons pas eu le temps d'intégrer proprement, serait d'utiliser encore plus cette clé entre deux itérations de recherche. Quand l'approfondissement passe de la profondeur d à la profondeur $d+1$, beaucoup de positions déjà vues réapparaissent. Avec la clé Zobrist, on peut les reconnaître immédiatement et vérifier si les informations de

l'itération précédente restent utiles: meilleur coup à tester en premier, borne alpha–beta, ou score déjà connu à une profondeur suffisante. L'idée est donc de ne pas seulement utiliser Zobrist comme index de table, mais aussi comme moyen de suivre les mêmes positions d'une itération à l'autre.

Listing 7: Mise à jour incrémentale de la clé Zobrist après un coup

```
static inline Board applyMove(const Board &b, const Move &mv) {
    Board child = b;
    Uint16 player = b.turn;
    Bitboard &me = (player == RED) ? child.red : child.blue;
    Bitboard &opp = (player == RED) ? child.blue : child.red;
    const Uint64 *myKeys = (player == RED) ? zobristRed : zobristBlue;

    Bitboard toMask = mask(mv.to);
    if (validJumps[mv.from] & toMask) {
        me &= ~mask(mv.from);
        child.key ^= myKeys[mv.from];
    }

    me |= toMask;
    child.key ^= myKeys[mv.to];

    Bitboard flips = validClones[mv.to] & opp;
    uint64_t flipIndex = _pext_u64(flips, validClones[mv.to]);
    child.key ^= zobristFlipTable[mv.to][flipIndex];

    opp &= ~flips;
    me |= flips;
    child.turn = opponent(player);
    child.key ^= zobristTurn;
    return child;
}
```

9 Cas limites

Aucun coup légal. Il arrive qu'un joueur ne puisse pas jouer. Dans ce cas, on ne doit pas inventer un coup: on passe simplement au joueur suivant. Si les deux joueurs sont bloqués, la partie est terminée et on compare les scores.

Cycles. Les stratégies trop simples peuvent parfois rejouer les mêmes types de coups et laisser la partie tourner en rond. Chercher plus profond limite ce problème, car l'algorithme voit mieux les conséquences de ses coups. Les tables de transposition aident aussi, puisqu'elles gardent en mémoire des positions déjà vues pendant la recherche.

Fatality et stalling. Dans le code, la fonction `fatality` sert à donner une valeur très claire aux positions vraiment terminales: une victoire finale doit être préférée à un simple bon score intermédiaire, et une défaite finale doit être reconnue comme mauvaise. Cela évite que l'algorithme choisisse une ligne jolie à court terme alors qu'elle mène à une fin perdante.

Il y a aussi un cas plus subtil: si la recherche voit surtout des lignes perdantes, finir vite n'a aucun intérêt. Dans ce cas, on préfère parfois jouer un coup qui garde la partie ouverte, voire qui ramène vers une boucle possible. Ce comportement de *stalling* n'est pas là pour inventer un coup illégal; il sert simplement à chercher une nulle ou une erreur adverse quand la position est déjà très mauvaise.

Une optimisation possible serait de traiter ce cas de manière plus explicite: garder une ligne de répétition comme coup de secours, puis lancer en parallèle d'autres recherches pour vérifier s'il existe une meilleure sortie. Si aucun thread ne trouve de coup réellement meilleur dans le temps disponible, on conserve la répétition; sinon, on bascule vers la ligne qui permet encore de jouer pour le gain.

La figure ci-dessous illustre ce type de situation. Le bleu peut se contenter de faire des sauts entre le coin droit et la case libre actuelle. Le rouge, lui, a la main sur la suite: soit il entretient la répétition, soit il tente un saut dans l'ouverture bleue au moment où elle apparaît. L'idée était donc de garder la répétition comme option sûre, tout en laissant un thread explorer en parallèle le saut risqué pour savoir, à plus grande profondeur, s'il existe une vraie sortie.



Figure 1: Exemple de position où le stalling peut servir de coup de secours pendant qu'une recherche parallèle étudie une sortie plus ambitieuse.

Les deux captures suivantes montrent un autre cas typique. Le rouge est déjà en mauvaise position. Une stratégie naïve qui ne regarde que le score matériel peut chercher à remplir les dernières cases, parce que les scores intermédiaires semblent meilleurs: par exemple $36-26$, puis $36-27$, puis $26-28$. Mais si cette ligne mène à une défaite terminale à -8, elle reste moins bonne qu'une ligne à -10 qui ne perd pas tout de suite et peut encore boucler. C'est pour cela que les positions terminales perdantes reçoivent un score très négatif: l'algorithme comprend qu'une répétition défavorable est préférable à une défaite certaine.

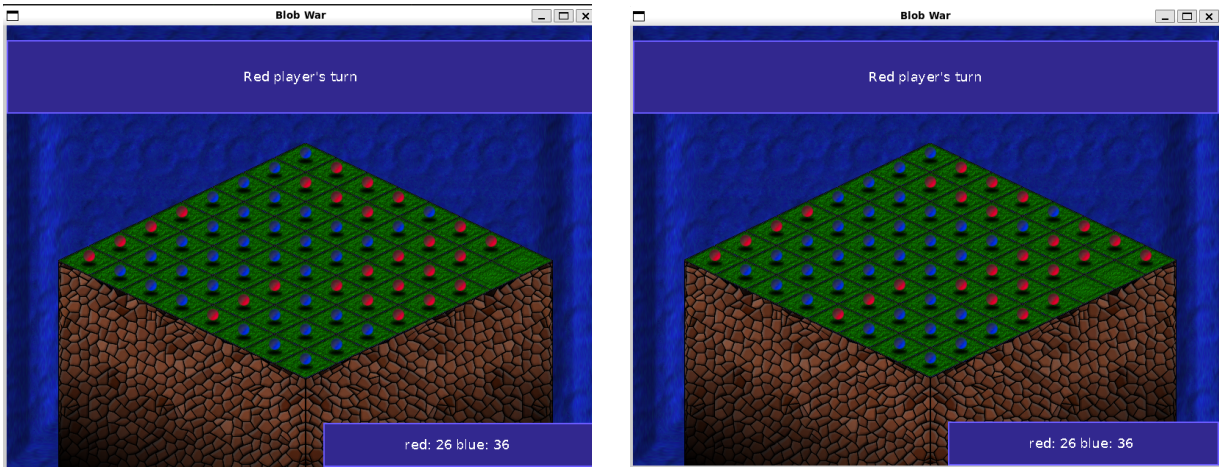


Figure 2: Deux positions d'une boucle où le rouge préfère répéter avec un mauvais score plutôt que remplir le plateau et perdre définitivement.

10 Analyse théorique

On note n **le nombre moyen de coups possibles**, d **la profondeur de recherche** et p **le nombre de cœurs CPU utilisés**. Sur Blobwar, n varie fortement selon la phase: il est faible au début, augmente lorsque le plateau se remplit de coups possibles, puis baisse en fin de partie lorsque les cases libres disparaissent. Les valeurs observées sur les positions de tournoi restent souvent dans un ordre de grandeur de quelques dizaines de coups, ce qui rend la recherche exhaustive complète impossible dès que d augmente.

Algorithme	Travail total T_1	Profondeur critique T_∞	Mémoire
Glouton	$\mathbf{O(n)}$	$O(1)$ si les coups sont parallélisés	$O(n)$
Minimax	$\mathbf{O(n^d)}$	$O(d)$ avec parallélisme idéal par niveau	$O(d)$ récursif
Alpha–beta, pire cas	$\mathbf{O(n^d)}$	$O(d)$ à ordre parfait théorique	$O(d)$ récursif
Alpha–beta, ordre idéal	$\mathbf{O(n^{d/2})}$	$O(d)$	$O(d)$ récursif
Minimax parallèle racine	$\mathbf{O(n^d)}$	$\mathbf{O(n^{d-1})}$	$O(pd)$
Alpha–beta parallèle racine	entre $\mathbf{O(n^{d/2})}$ et $\mathbf{O(n^d)}$	dépend des bornes partagées	$O(pd)$
Alpha–beta + TT	$\mathbf{O(P_d)}$ positions distinctes	dépend des accès à la table	$O(\|TT\|)$

Table 2: Complexité théorique. T_1 correspond au travail total. T_∞ correspond à la partie qui reste forcément séquentielle, même avec beaucoup de processeurs.

Glouton. Le glouton ne développe qu'un seul niveau. Son travail est linéaire dans le nombre de coups légaux: il applique chaque coup une fois et appelle la fonction d'évaluation. La profondeur critique est très faible, car les évaluations des coups sont indépendantes. C'est donc un algorithme facile à paralléliser, mais il reste limité parce qu'il ne regarde qu'un coup en avance.

Minimax. Minimax visite tout l'arbre jusqu'à profondeur d. Son travail est

$$\boxed{\mathbf{T_1(d) = 1 + n + n^2 + \cdots + n^d = O(n^d)}}.$$

La pile de récursion a une taille $O(d)$. Même avec beaucoup de threads, il faut toujours suivre au moins un chemin de longueur d. Dans notre cas, la parallélisation se fait surtout à la racine: le travail reste $O(n^d)$, mais il est partagé entre les cœurs.

Alpha–beta. Alpha–beta ne change pas la valeur calculée par minimax, seulement le nombre de nœuds visités. Dans le pire cas, si les coups sont explorés dans le plus mauvais ordre, aucune coupure utile n'est obtenue et la complexité reste $O(n^d)$. Dans le cas idéal, le meilleur coup est toujours exploré en premier et la complexité tombe autour de $O(n^{d/2})$. C'est pour cela que l'ordre des coups est très important: si les bons coups sont essayés tôt, on coupe beaucoup plus.

Travail et profondeur en parallèle. Pour un algorithme parallèle, on peut résumer le temps idéal avec cette borne:

$$\boxed{\mathbf{T_p} \geq \max\left(\frac{\mathbf{T_1}}{\mathbf{p}}, \mathbf{T_\infty}\right)}.$$

Le gain est donc limité par deux choses: le travail total à faire et la partie qui ne se parallélise pas bien. Pour minimax, la répartition est simple car les sous-arbres de la racine sont presque indépendants. Pour alpha–beta, c'est plus compliqué: si on explore trop de branches en même temps, les bonnes bornes arrivent trop tard et on coupe moins. C'est pour cela qu'on cherche d'abord un premier coup, puis qu'on partage une borne atomique et une table de transposition entre les threads.

Tables de transposition. La table de transposition évite de recalculer plusieurs fois une même position. Au lieu de raisonner seulement en nombre de chemins dans l'arbre, on se rapproche du nombre de positions différentes réellement rencontrées, noté P_d. On a toujours $P_d \leq n^d$. En échange, on utilise plus de mémoire, et les collisions dans la table peuvent faire perdre de l'information. C'est pour cela qu'on garde en priorité les entrées calculées le plus profondément.

Anticipation du coup adverse. L'anticipation ne réduit pas la complexité d'une recherche spécifique. Son intérêt est d'exploiter le temps pendant lequel l'adversaire réfléchit. Lors du tour adverse on lance une recherche comme si c'était notre tour. Si le coup adverse est correctement prédit, on continue simplement à travailler, on a doublé notre temps de réflexion. Dans le cas contraire, la recherche spéculative est abandonnée et on commence normalement le nouveau tour.

Pour les calculs il faut impérativement distinguer deux métriques : Le *hit brut* indique que le coup adverse a été correctement prédit. Le *useful hit* indique que cette prédiction a réellement permis de terminer une itération durant la seconde bonus. Ce second critère est plus important, car une prédiction correcte ne suffit pas toujours : si la recherche anticipée ne termine pas une nouvelle profondeur exploitable, elle n'améliore pas directement le coup joué.

En notant h le taux de hit brut et q la proportion de hits bruts qui deviennent des *useful hits*, le temps de recherche effectivement exploitable peut être approximé par :

$$\boxed{T_{\text{utile}} \approx T_{\text{tour}} + h \cdot q \cdot T_{\text{adverse}}}.$$

Dans nos logs, les meilleures versions prédisent correctement le coup adverse environ une fois sur deux. Parmi ces prédictions correctes, environ la moitié devient réellement utile, ce qui donne un taux global d'*useful hits* de l'ordre de 25 à 30%.

L'effet est cumulatif : plus les prédictions sont fréquentes, plus le moteur dispose de temps effectif pour stabiliser sa variation principale et atteindre des profondeurs plus élevées. Dans plusieurs parties, l'anticipation permettait de gagner un ou deux niveaux de profondeur par rapport à une recherche limitée au seul temps de notre tour. Comme tous les moteurs reposent sur l'iterative deepening, avoir une profondeur d'avance n'assure pas un gain régulier (on passe juste à des victoires *plus probables*), en revanche le fait d'obetnir parfois une $2_{ème}$ depth d'avance grâce au shortcut change totalement la donne, cela signifie tout simplement qu'on réfléchit plus profond à **notre** tour, que l'adversaire pendant **le sien**. Quand ça arrive les probabilités de victoire s'envolent.

11 Protocole expérimental

Nous avons lancé des benchmarks entre les différentes versions de nos algorithmes. Au total, cela représente 4 973 parties: 2 587 sur `standard` et 2 386 sur `x`. Chaque paire d'algorithmes est jouée sur plusieurs parties, avec alternance des couleurs.

Les résultats détaillés sont disponibles sur notre site: https://makh.fr/blobwar/
Nous avons surtout regardé:

- taux de victoire, nul et défaite;
- score final moyen;
- profondeur atteinte;
- nombre de nœuds explorés par seconde, noté NPS dans les tableaux;
- taux de hit de la table de transposition;
- taux de coupures alpha–beta;
- qualité du tri des coups et taux de réussite de l'anticipation.

Stratégie	V	N	D	Parties	Score	Marge	Prof.	NPS	Shortcut
NMP_TT_SF_v3_15	**109**	**53**	**82**	**244**	**55.5%**	**+0.66**	**7.0**	**83.2M**	**58.2%**
NMP_TT_SF_v3	**622**	**323**	**489**	**1434**	**54.6%**	**+0.63**	**7.0**	**89.3M**	**58.8%**
NMP_TT_SF_v7	136	66	116	318	53.1%	+0.46	7.0	91.1M	58.9%
NMP_TT_SF_v5	387	205	348	940	52.1%	+0.34	7.0	87.1M	64.8%
NMP_TT_SF_v1	128	76	136	340	48.8%	-0.30	7.0	93.9M	–
NMP_TT_SF_v4	74	34	86	194	46.9%	-0.44	7.0	55.9M	59.6%
NMP_TT_SF_v5_15	137	83	165	385	46.4%	-0.45	7.0	84.6M	64.6%
NMP_TT_v2	364	230	446	1040	46.1%	-0.43	7.0	100.4M	–
NMP_TT_SF_v2	39	21	84	144	34.4%	-2.43	7.0	57.0M	–

Table 3: Classement agrégé sur la carte `standard`. Le score compte une victoire comme 1 point et une nulle comme 0.5. Le tableau reprend les séries les plus exploitables.

Stratégie	V	N	D	Parties	Score	Marge	Prof.	NPS	Shortcut
NMP_TT_SF_v5	**253**	**195**	**197**	**645**	**54.3%**	**+0.33**	**9.0**	**55.2M**	**69.3%**
NMP_TT_SF_v7	**177**	**130**	**151**	**458**	**52.8%**	**+0.00**	**8.5**	**46.6M**	**70.0%**
NMP_TT_SF_v3	619	466	554	1639	52.0%	+0.33	9.0	58.6M	67.5%
NMP_TT_SF_v5_15	90	76	92	258	49.6%	+0.24	9.0	57.9M	68.2%
NMP_TT_SF_v1	123	113	144	380	47.2%	-0.47	8.0	50.2M	–
NMP_TT_v2	308	265	369	942	46.8%	-0.33	8.0	53.8M	–
NMP_TT_SF_v4	65	65	88	218	44.7%	-0.53	8.0	34.5M	66.2%
NMP_TT_SF_v2	36	32	52	120	43.3%	-1.05	8.0	32.5M	–

Table 4: Classement agrégé sur la carte `x`.

Stratégie	Carte	Hit TT	Cutoff TT	Coupure après tri	1er coup coupe
NMP_TT_v2	standard	10.1%	2.6%	8.5%	92.6%
NMP_TT_SF_v3	standard	7.8%	2.9%	13.2%	94.1%
NMP_TT_SF_v5	standard	11.1%	2.8%	10.2%	93.1%
NMP_TT_SF_v7	standard	10.3%	2.9%	11.1%	93.6%
NMP_TT_v2	x	10.3%	5.0%	14.4%	93.3%
NMP_TT_SF_v3	x	5.6%	3.3%	11.7%	93.4%
NMP_TT_SF_v5	x	9.6%	3.7%	11.9%	93.1%
NMP_TT_SF_v7	x	10.7%	4.0%	12.2%	93.0%

Table 5: Mesures internes de recherche. Le taux de hit TT ne suffit pas à lui seul: le tri des coups et l'anticipation jouent aussi beaucoup.

12 Résultats qualitatifs

Les mesures du tableau de bord font ressortir plusieurs points.

- Le glouton joue instantanément mais se piège facilement.
- Minimax améliore la qualité des coups, mais sa profondeur reste limitée.
- Alpha–beta permet d'atteindre une profondeur supérieure à temps égal, surtout avec un bon ordre des coups.
- Les versions avec mémoire entre les tours et anticipation sont les plus régulières: sur `standard`, `NMP_TT_SF_v3_15`, `v3`, `v7` et `v5` sont toutes au-dessus de 52% de score.
- La carte influence fortement le classement. `NMP_TT_SF_v5` domine sur `x`, alors que `NMP_TT_SF_v3_15` arrive en tête sur `standard`.
- Le NPS brut ne suffit pas à prédire la force. `NMP_TT_v2` a un NPS très élevé sur `standard`, mais reste sous 50% de score. La qualité du tri des coups, de l'anticipation et des positions explorées compte davantage.
- L'anticipation est utile: les meilleures versions devinent environ 58–70% des coups selon la carte, et le pourcentage de victoire suit surtout la fréquence des *useful hits*. Nous pensons que cette amélioration a été l'un des facteurs clef de notre victoire au tournoi.
- Le parallélisme CPU augmente la profondeur atteinte, mais le gain n'est pas linéaire: certaines versions parallèles sont moins fortes que la version de base correspondante, ce qui montre le coût des pertes de coupures et de la pression mémoire.

13 Conclusion

Nous avons programmé les algorithmes demandés progressivement, depuis une base gloutonne jusqu'à des recherches alpha–beta parallèles. Les améliorations les plus importantes viennent de la représentation bitboard, de l'ordre des coups, de l'iterative deepening, des tables de transposition et de l'anticipation.

Il est aussi notable que, même si nos pistes étaient conceptuelles (nous n'avons pas essayé de mettre en place une approche basée sur le *machine learning*), leur validation fut presque toujours empirique. Ce sont les données récoltées qui nous ont permis de savoir quel algorithme était prometteur, ou quelle optimisation semblait intéressante. Dire que les résultats ont été contre-intuitifs serait un euphémisme.

Deux points nous ont particulièrement surpris. D'abord, le parallélisme n'est pas naturellement meilleur que le séquentiel, ensuite, les résultats ne sont pas nécessairement meilleurs quand on conserve la même mémoire tout au long de la partie. Dans les deux cas au départ c'est même plutôt l'inverse.